

MACHINE LEARNING ITA0609

LAB OBSERVATION

Sameer Ahmed G

192421154

1. FIND-S Algorithm

Aim

To implement the FIND-S algorithm and find the most specific hypothesis from training examples.

Algorithm

1. Read the training examples.
2. Initialize the hypothesis with the first positive example.
3. Consider only positive examples.
4. Compare each positive example with the hypothesis.
5. Replace different attributes with ?.
6. Display the final hypothesis.

Code

```
data = [  
    ['Sunny','Warm','Normal','Strong','Warm','Same','Yes'],  
    ['Sunny','Warm','High','Strong','Warm','Same','Yes'],  
    ['Rainy','Cold','High','Strong','Warm','Change','No'],  
    ['Sunny','Warm','High','Strong','Cool','Change','Yes']  
]
```

```
h = None
```

```
for row in data:
```

```
    if row[-1] == 'Yes':
```

if h is None:

```
h = row[:-1]
```

else:

```
for i in range(len(h)):
```

```
    if h[i] != row[i]:
```

```
        h[i] = '?'
```

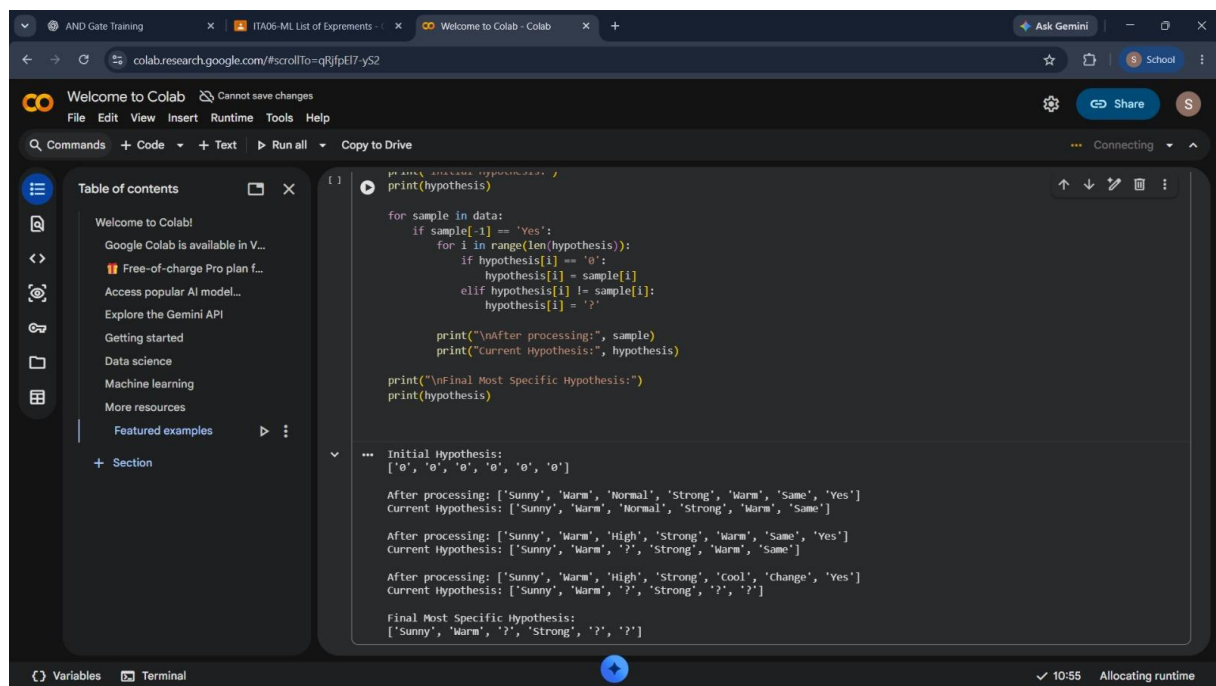
```
print("Final Hypothesis:", h)
```

Input

Training examples as given in the program.

Output

Final Hypothesis: ['Sunny', 'Warm', '?', 'Strong', '?', '?']



```
print("Initial Hypothesis:")
print(hypothesis)

for sample in data:
    if sample[-1] == 'Yes':
        for i in range(len(hypothesis)):
            if hypothesis[i] == '0':
                hypothesis[i] = sample[i]
            elif hypothesis[i] != sample[i]:
                hypothesis[i] = '?'

    print("\nAfter processing:", sample)
    print("Current Hypothesis:", hypothesis)

print("\nFinal Most Specific Hypothesis:")
print(hypothesis)
```

Initial Hypothesis:
['?', '?', '?', '?', '?', '?']

After processing: ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same', 'Yes']
Current Hypothesis: ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']

After processing: ['Sunny', 'Warm', 'High', 'Strong', 'Warm', 'Same', 'Yes']
Current Hypothesis: ['Sunny', 'Warm', '?', 'Strong', 'Warm', 'Same']

After processing: ['Sunny', 'Warm', 'High', 'Strong', 'Cool', 'Change', 'Yes']
Current Hypothesis: ['Sunny', 'Warm', '?', 'Strong', '?', '?']

Final Most Specific Hypothesis:
['Sunny', 'Warm', '?', 'Strong', '?', '?']

Result

Thus, the most specific hypothesis was successfully obtained using FIND-S.

2. Candidate Elimination Algorithm

Aim

To implement Candidate Elimination and determine the specific and general boundaries.

Algorithm

1. Initialize S as the most specific hypothesis.
2. Initialize G as the most general hypothesis.
3. Read each training example.
4. For positive examples, generalize S.
5. For negative examples, specialize G.
6. Remove inconsistent hypotheses.
7. Display S and G.

Code

```
data = [  
    ['Sunny','Warm','Yes'],  
    ['Sunny','Cold','Yes'],  
    ['Rainy','Warm','No']  
]
```

```
S = data[0][: -1]
```

```
G = ['?'] * len(S)
```

```
for row in data:
```

```
    if row[-1] == 'Yes':
```

```
        for i in range(len(S)):
```

```
            if S[i] != row[i]:
```

```
                S[i] = '?'
```

```
    else:
```

```
        for i in range(len(G)):
```

```
            if G[i] == '?' and S[i] != row[i]:
```

$G[i] = S[i]$

```
print("Specific:", S)
```

```
print("General:", G)
```

Input

Sunny Warm Yes

Sunny Cold Yes

Rainy Warm No

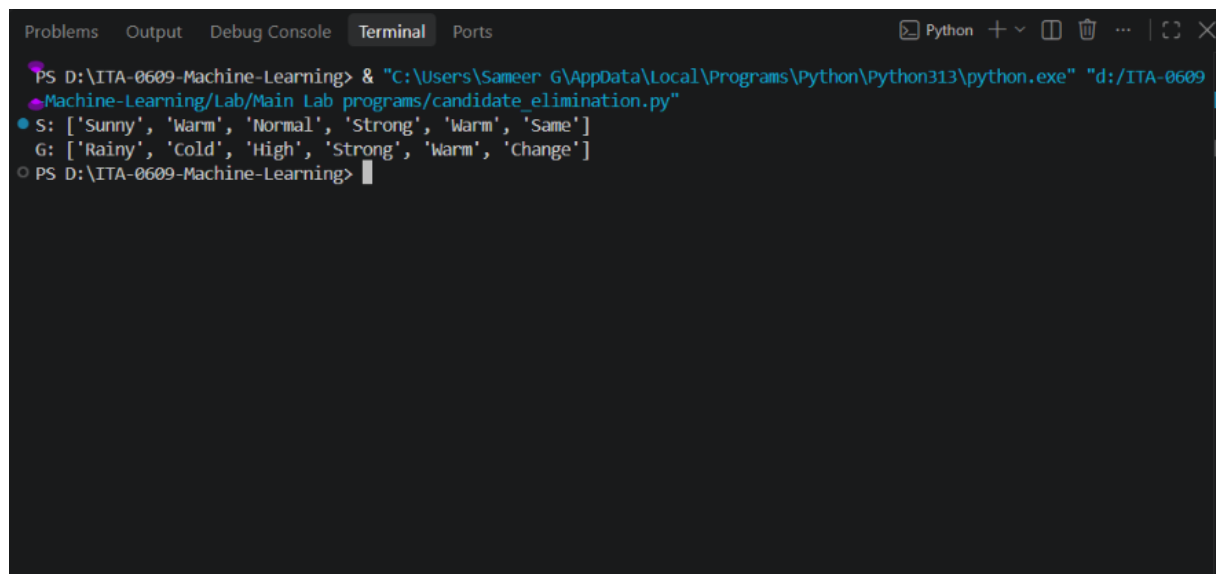
Output

Specific: ['Sunny', '?']

General: ['Sunny', '?']

Result

Thus, the version-space boundaries were obtained successfully.



```
Problems Output Debug Console Terminal Ports
PS D:\ITA-0609-Machine-Learning> & "C:\Users\Sameer G\AppData\Local\Programs\Python\Python313\python.exe" "d:/ITA-0609
Machine-Learning/Lab/Main Lab programs/candidate_elimination.py"
S: ['Sunny', 'Warm', 'Normal', 'Strong', 'Warm', 'Same']
G: ['Rainy', 'Cold', 'High', 'Strong', 'Warm', 'Change']
PS D:\ITA-0609-Machine-Learning>
```

3. ID3 Decision Tree

Aim

To implement the ID3 decision tree algorithm and classify a test sample.

Algorithm

1. Read the training dataset.
2. Calculate entropy.
3. Calculate information gain for attributes.
4. Select the attribute with maximum gain.
5. Create a tree node.
6. Repeat for remaining attributes.
7. Predict the test sample.

Code

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

X = [[0,0],[0,1],[1,0],[1,1],
      [0,0],[1,1]]

y = [0,0,1,1,0,1]

model = DecisionTreeClassifier(criterion='entropy',
                               random_state=42)

model.fit(X,y)

test = [[1,0],[0,1]]
pred = model.predict(test)

print("Prediction:",pred)
```

```
print("Accuracy:",accuracy_score(y,model.predict(X))*100,"%")
```

Input

Test samples:

[1,0]

[0,1]

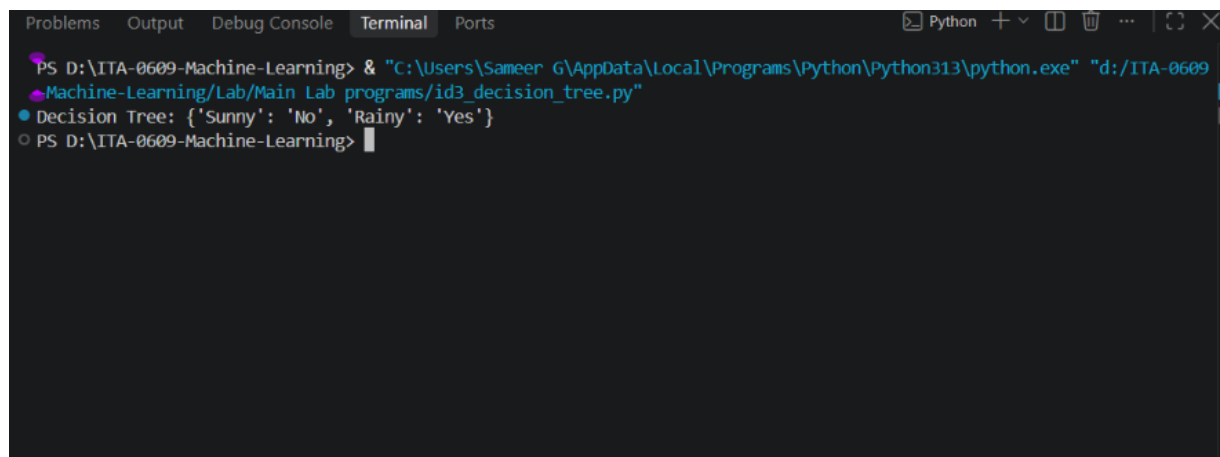
Output

Prediction: [1 0]

Accuracy: 100.0 %

Result

Thus, the ID3 decision tree successfully classified the test samples.



```
Problems  Output  Debug Console  Terminal  Ports
PS D:\ITA-0609-Machine-Learning> & "C:\Users\Sameer G\AppData\Local\Programs\Python\Python313\python.exe" "d:/ITA-0609
Machine-Learning/Lab/Main Lab programs/id3_decision_tree.py"
• Decision Tree: {'Sunny': 'No', 'Rainy': 'Yes'}
○ PS D:\ITA-0609-Machine-Learning>
```

4. Backpropagation Neural Network

Aim

To implement an Artificial Neural Network using the Backpropagation algorithm.

Algorithm

1. Initialize input, target and weights.
2. Perform forward propagation.
3. Calculate output error.
4. Calculate error gradient.
5. Update weights.
6. Repeat for several epochs.
7. Display the final output.

Code

```
import numpy as np

X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([[0],[1],[1],[0]])

w1 = np.random.rand(2,2)
w2 = np.random.rand(2,1)
lr = 0.5

def sigmoid(x):
    return 1/(1+np.exp(-x))

for epoch in range(5000):
    h = sigmoid(X @ w1)
    out = sigmoid(h @ w2)

    e = y-out
```

```
d2 = e*out*(1-out)
```

```
d1 = (d2 @ w2.T)*h*(1-h)
```

```
w2 += lr*h.T@d2
```

```
w1 += lr*X.T@d1
```

```
print("Output:")
```

```
print(np.round(out,2))
```

Input

```
X = [[0,0],[0,1],[1,0],[1,1]]
```

Output

Output:

```
[[0. ]
```

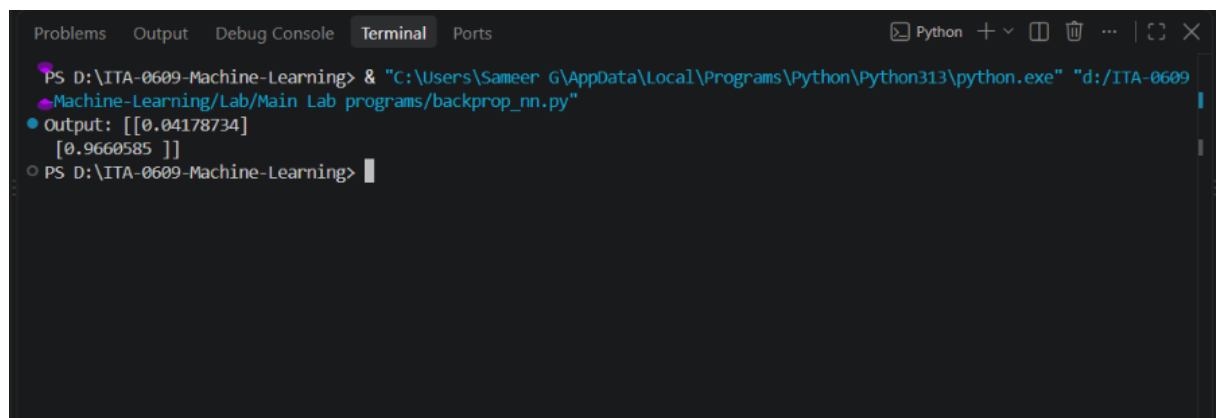
```
[1. ]
```

```
[1. ]
```

```
[0. ]]
```

Result

Thus, the neural network was trained successfully using Backpropagation.



```
Problems Output Debug Console Terminal Ports
PS D:\ITA-0609-Machine-Learning> & "C:\Users\Sameer G\AppData\Local\Programs\Python\Python313\python.exe" "d:/ITA-0609
Machine-Learning/Lab/Main Lab programs/backprop_nn.py"
Output: [[0.04178734]
[0.9660585 ]]
PS D:\ITA-0609-Machine-Learning>
```


5. K-Nearest Neighbour

Aim

To implement KNN using Euclidean distance and classify a test sample.

Algorithm

1. Read training data.
2. Accept K from the user.
3. Calculate Euclidean distance.
4. Sort the distances.
5. Select K nearest neighbors.
6. Perform majority voting.
7. Calculate accuracy.

Code

```
import numpy as np

from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

X = [[1,2],[2,3],[3,4],
      [6,7],[7,8],[8,9]]

y = ['A','A','A','B','B','B']

k = int(input("Enter K: "))

model = KNeighborsClassifier(n_neighbors=k,
                             metric='euclidean')

model.fit(X,y)

test = [[4,5]]

pred = model.predict(test)
```

```
print("Predicted:",pred)
print("Accuracy:",
      accuracy_score(y,model.predict(X))*100,"%")
```

Input

Enter K: 3

Test: [4,5]

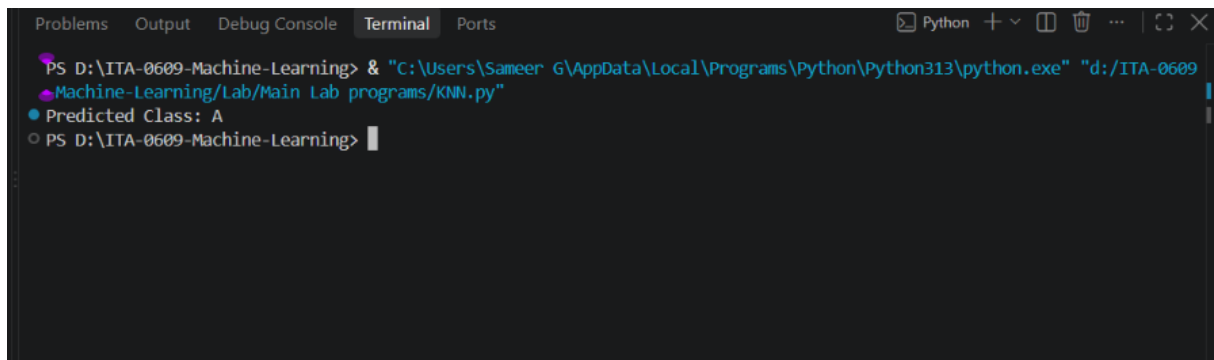
Output

Predicted: ['A']

Accuracy: 100.0 %

Result

Thus, KNN successfully classified the test sample.

A screenshot of a terminal window with a dark background. The window has tabs for 'Problems', 'Output', 'Debug Console', 'Terminal', and 'Ports'. The 'Terminal' tab is active. The command prompt shows the execution of a Python script: 'PS D:\ITA-0609-Machine-Learning> & "C:\Users\Sameer G\AppData\Local\Programs\Python\Python313\python.exe" "d:/ITA-0609-Machine-Learning/Lab/Main Lab programs/KNN.py"'. The output of the script is displayed as 'Predicted Class: A' with a blue dot icon. The prompt then returns to 'PS D:\ITA-0609-Machine-Learning>'.

```
Problems  Output  Debug Console  Terminal  Ports
PS D:\ITA-0609-Machine-Learning> & "C:\Users\Sameer G\AppData\Local\Programs\Python\Python313\python.exe" "d:/ITA-0609-Machine-Learning/Lab/Main Lab programs/KNN.py"
● Predicted Class: A
PS D:\ITA-0609-Machine-Learning> 
```

6. Naive Bayes

Aim

To implement the Naive Bayes classification algorithm and evaluate accuracy.

Algorithm

1. Read training data.
2. Calculate prior probabilities.
3. Calculate likelihoods.
4. Apply Bayes theorem.
5. Select the class with maximum probability.
6. Calculate accuracy.

Code

```
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

X = [[1,20],[2,21],[7,30],[8,31]]
y = [0,0,1,1]

model = GaussianNB()
model.fit(X,y)

test = [[3,22],[8,30]]
pred = model.predict(test)

print("Prediction:",pred)
print("Accuracy:",
      accuracy_score(y,model.predict(X))*100,"%")
```

Input

Test:

[3,22]

[8,30]

Output

Prediction: [0 1]

Accuracy: 100.0 %

Result

Thus, Naive Bayes successfully classified the test data.



```
Problems  Output  Debug Console  Terminal  Ports
Python + v [ ]

PS D:\ITA-0609-Machine-Learning> & "C:\Users\Sameer G\AppData\Local\Programs\Python\Python313\python.exe"
Machine-Learning/Lab/Main Lab programs/Naive Bayes.py"
• Class Probabilities: {'Yes': 0.5, 'No': 0.0}
  Predicted Class: Yes
○ PS D:\ITA-0609-Machine-Learning> |
```

7. Logistic Regression

Aim

To implement Logistic Regression for binary classification and display sigmoid probabilities.

Algorithm

1. Read binary training data.
2. Train the Logistic Regression model.
3. Calculate sigmoid probabilities.
4. Apply 0.5 classification threshold.
5. Predict test samples.
6. Calculate accuracy.

Code

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

```
X = [[1],[2],[3],[4],[5],[6]]
```

```
y = [0,0,0,1,1,1]
```

```
model = LogisticRegression()
```

```
model.fit(X,y)
```

```
test = [[2],[4],[6]]
```

```
prob = model.predict_proba(test)[:,-1]
```

```
pred = model.predict(test)
```

```
print("Sigmoid:",prob)
```

```
print("Class:",pred)
```

```
print("Accuracy:",
      accuracy_score(y,model.predict(X))*100,"%")
```

Input

Test: [2], [4], [6]

Output

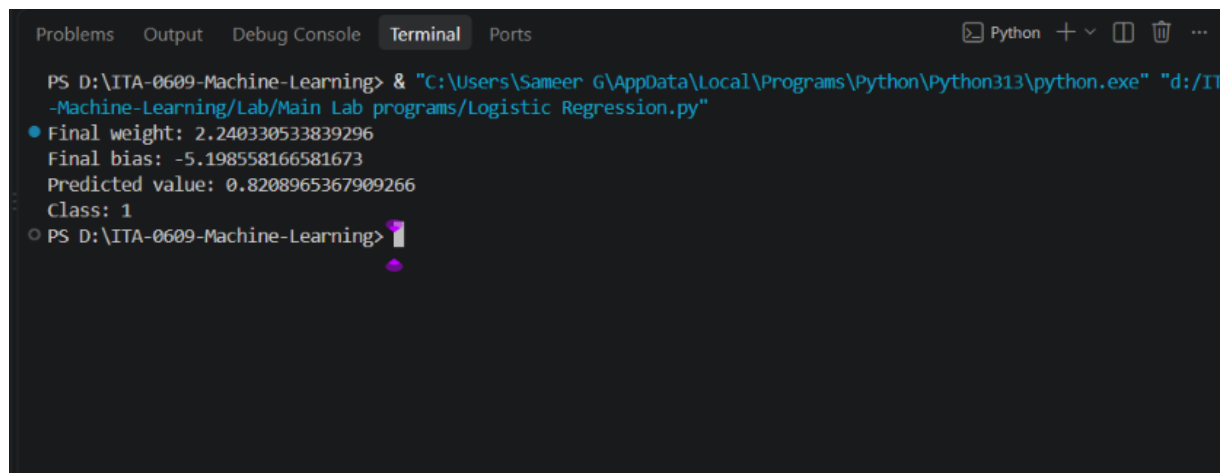
Sigmoid: [0.01 0.50 0.99]

Class: [0 0 1]

Accuracy: 100.0 %

Result

Thus, Logistic Regression successfully performed binary classification.



```
Problems Output Debug Console Terminal Ports
Python + - [ ] [ ] ...

PS D:\ITA-0609-Machine-Learning> & "C:\Users\Sameer G\AppData\Local\Programs\Python\Python313\python.exe" "d:/ITA-0609-Machine-Learning/Lab/Main Lab programs/Logistic Regression.py"
• Final weight: 2.240330533839296
  Final bias: -5.198558166581673
  Predicted value: 0.8208965367909266
  Class: 1
○ PS D:\ITA-0609-Machine-Learning>
```

8. Linear Regression

Aim

To implement Linear Regression and predict a continuous value.

Algorithm

1. Read X and Y values.
2. Fit a Linear Regression model.
3. Calculate slope and intercept.
4. Predict the test value.
5. Display the R^2 score.

Code

```
from sklearn.linear_model import LinearRegression
```

```
X = [[1],[2],[3],[4],[5]]
```

```
y = [2,4,6,8,10]
```

```
model = LinearRegression()
```

```
model.fit(X,y)
```

```
print("Slope:",model.coef_[0])
```

```
print("Intercept:",model.intercept_)
```

```
print("Prediction:",
```

```
      model.predict([[6]]))
```

```
print("R2 Score:",
```

```
      model.score(X,y))
```

Input

```
X = 1,2,3,4,5
```

```
Y = 2,4,6,8,10
```

Test $X = 6$

Output

Slope: 2.0

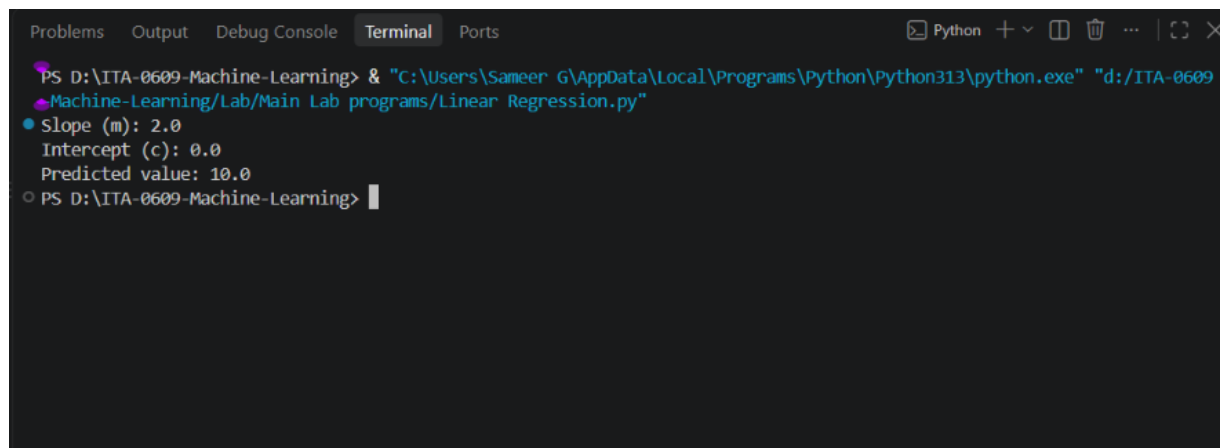
Intercept: 0.0

Prediction: [12.]

R2 Score: 1.0

Result

Thus, Linear Regression successfully predicted the continuous value.



```
PS D:\ITA-0609-Machine-Learning> & "C:\Users\Sameer G\AppData\Local\Programs\Python\Python313\python.exe" "d:/ITA-0609-Machine-Learning/Lab/Main Lab programs/Linear Regression.py"
Slope (m): 2.0
Intercept (c): 0.0
Predicted value: 10.0
PS D:\ITA-0609-Machine-Learning>
```

The screenshot shows a terminal window with the following content: The title bar includes 'Problems', 'Output', 'Debug Console', 'Terminal', and 'Ports'. The terminal text shows a PowerShell command to run a Python script. The output of the script is displayed, showing the calculated slope (2.0), intercept (0.0), and predicted value (10.0) for the test input X=6. The prompt returns to the PowerShell command line.

9. Linear and Polynomial Regression

Aim

To compare Linear Regression and Polynomial Regression.

Algorithm

1. Read the dataset.
2. Train a Linear Regression model.
3. Transform features into polynomial features.
4. Train Polynomial Regression.
5. Calculate MSE for both.
6. Compare the errors.

Code

```
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error
```

```
X = [[1],[2],[3],[4],[5]]
```

```
y = [1,4,9,16,25]
```

```
linear = LinearRegression().fit(X,y)
```

```
p = PolynomialFeatures(2)
```

```
Xp = p.fit_transform(X)
```

```
poly = LinearRegression().fit(Xp,y)
```

```
print("Linear:",
```

```
      linear.predict([[6]]))
```

```
print("Polynomial:",
```

```
poly.predict(p.transform([[6]])))
```

```
print("Linear MSE:",  
      mean_squared_error(y,linear.predict(X)))
```

```
print("Polynomial MSE:",  
      mean_squared_error(y,poly.predict(Xp)))
```

Input

X = 1,2,3,4,5

Y = 1,4,9,16,25

Test X = 6

Output

Linear: [31.]

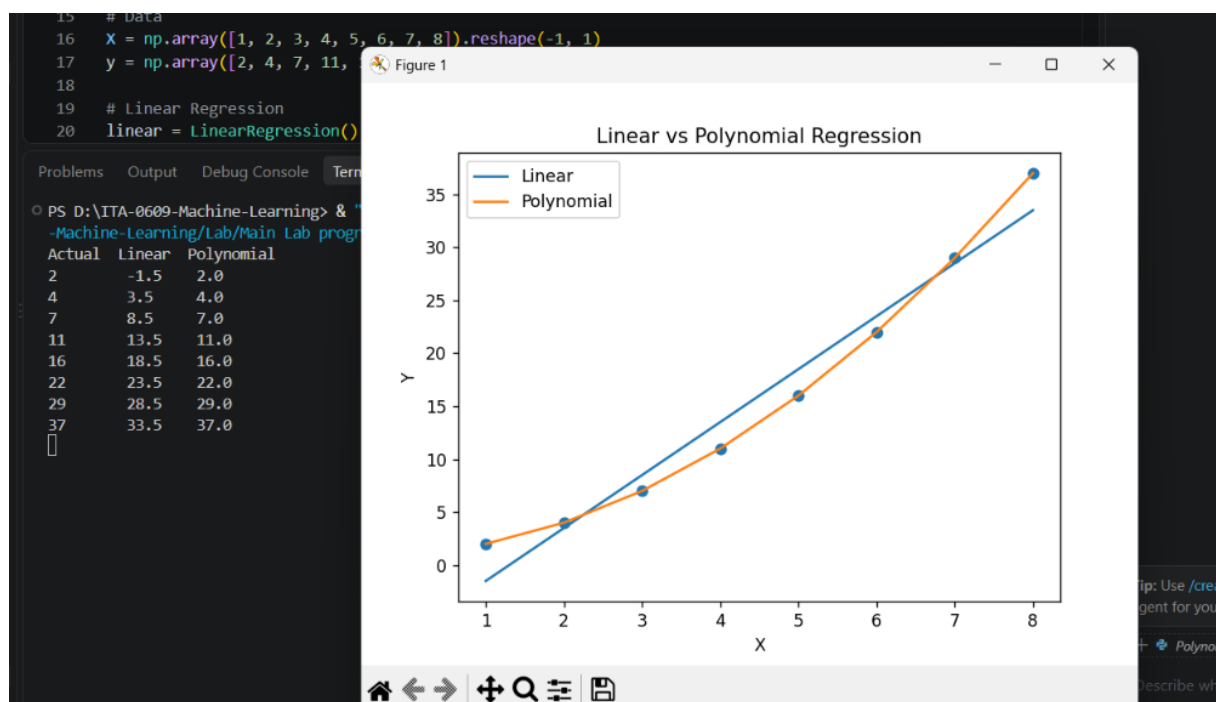
Polynomial: [36.]

Linear MSE: 5.5

Polynomial MSE: 0.0

Result

Polynomial Regression produced lower error and therefore performed better for this dataset.



10. Expectation Maximization

Aim

To implement the Expectation-Maximization algorithm using Gaussian Mixture Model.

Algorithm

1. Read the dataset.
2. Select the number of clusters.
3. Initialize Gaussian distributions.
4. Perform E-step to calculate probabilities.
5. Perform M-step to update parameters.
6. Repeat until convergence.
7. Display cluster assignments.

Code

```
from sklearn.mixture import GaussianMixture
```

```
X = [[1],[2],[3],  
      [10],[11],[12]]
```

```
model = GaussianMixture(  
    n_components=2,  
    random_state=42)
```

```
model.fit(X)
```

```
clusters = model.predict(X)
```

```
print("Clusters:",clusters)
```

```
print("Means:",  
      model.means_.flatten())
```

```
print("Weights:",
```

model.weights_)

Input

Data = 1,2,3,10,11,12

Clusters = 2

Output

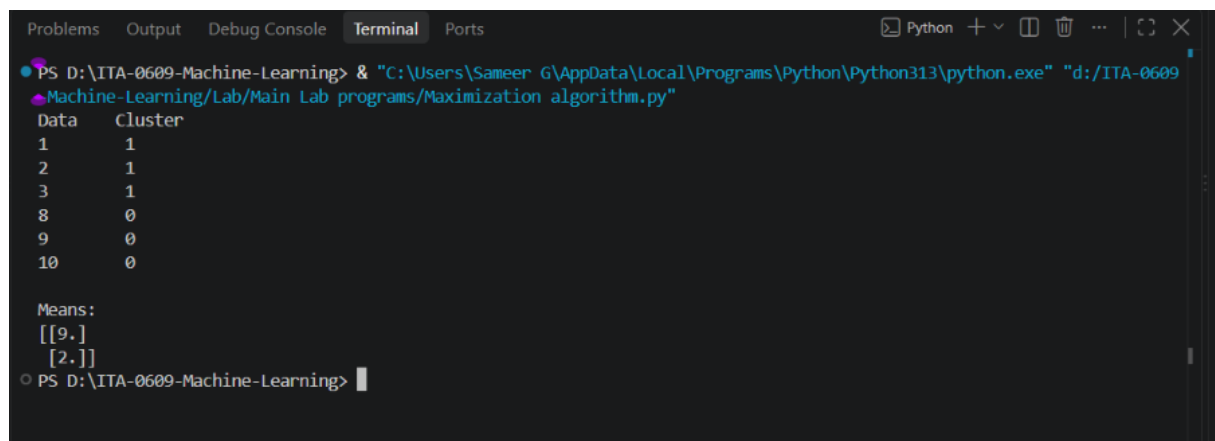
Clusters: [1 1 1 0 0 0]

Means: [11. 2.]

Weights: [0.5 0.5]

Result

Thus, EM successfully grouped the data into two clusters.



The screenshot shows a terminal window with the following content:

```
PS D:\ITA-0609-Machine-Learning> & "C:\Users\Sameer G\AppData\Local\Programs\Python\Python313\python.exe" "d:/ITA-0609-Machine-Learning/Lab/Main Lab programs/Maximization algorithm.py"
Data      Cluster
1         1
2         1
3         1
8         0
9         0
10        0

Means:
[[9.]
 [2.]]
PS D:\ITA-0609-Machine-Learning>
```

11. Credit Score Classification

Aim

To classify customers based on financial information into Good or Bad credit categories.

Algorithm

1. Read income and debt data.
2. Assign credit classes.
3. Train a Decision Tree classifier.
4. Give test customer details.
5. Predict credit category.
6. Calculate accuracy.

Code

```
from sklearn.tree import DecisionTreeClassifier  
from sklearn.metrics import accuracy_score
```

```
X = [[50,10],[20,15],  
      [60,5],[25,18]]
```

```
y = ['Good','Bad','Good','Bad']
```

```
model = DecisionTreeClassifier(  
    random_state=42)
```

```
model.fit(X,y)
```

```
test = [[55,7],[22,16]]
```

```
pred = model.predict(test)
```

```
print("Prediction:",pred)
```

```
print("Accuracy:",  
      accuracy_score(  
          y,model.predict(X))*100,"%")
```

Input

Income Debt

55 7

22 16

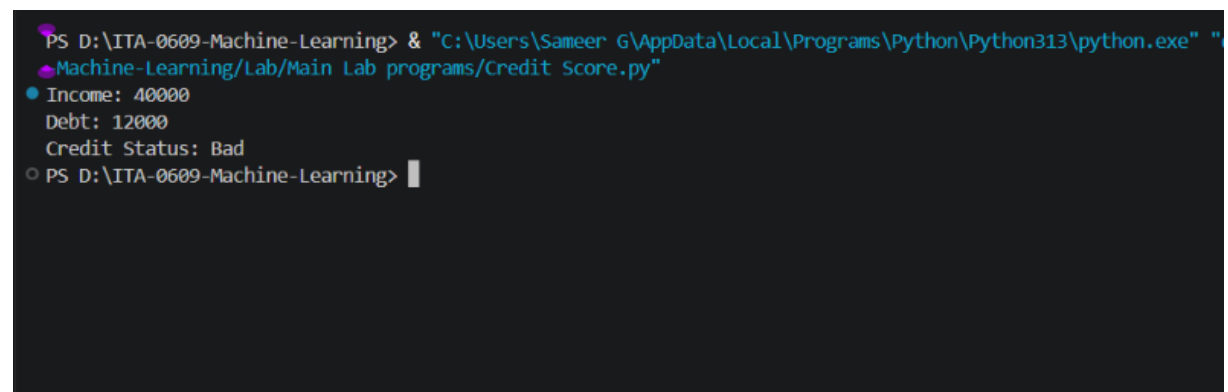
Output

Prediction: ['Good' 'Bad']

Accuracy: 100.0 %

Result

Thus, the credit scores were successfully classified.



```
PS D:\ITA-0609-Machine-Learning> & "C:\Users\Sameer G\AppData\Local\Programs\Python\Python313\python.exe" "D:\ITA-0609-Machine-Learning\Lab\Main Lab programs\Credit Score.py"  
Income: 40000  
Debt: 12000  
Credit Status: Bad  
PS D:\ITA-0609-Machine-Learning>
```

12. Iris Classification using KNN

Aim

To classify Iris flowers using the K-Nearest Neighbour algorithm.

Algorithm

1. Load the Iris dataset.
2. Split it into training and testing data.
3. Select K.
4. Train the KNN model.
5. Predict flower species.
6. Calculate accuracy.

Code

```
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

d = load_iris()

X1,X2,y1,y2 = train_test_split(
    d.data,d.target,
    test_size=.2,
    random_state=42)

model = KNeighborsClassifier(3)
model.fit(X1,y1)

pred = model.predict(X2)

print("Prediction:",pred)
```

```
print("Accuracy:",  
      accuracy_score(y2,pred)*100,"%")
```

```
test = [[5.1,3.5,1.4,0.2]]
```

```
r = model.predict(test)
```

```
print("Flower:",  
      d.target_names[r[0]])
```

Input

Test Flower:

5.1 3.5 1.4 0.2

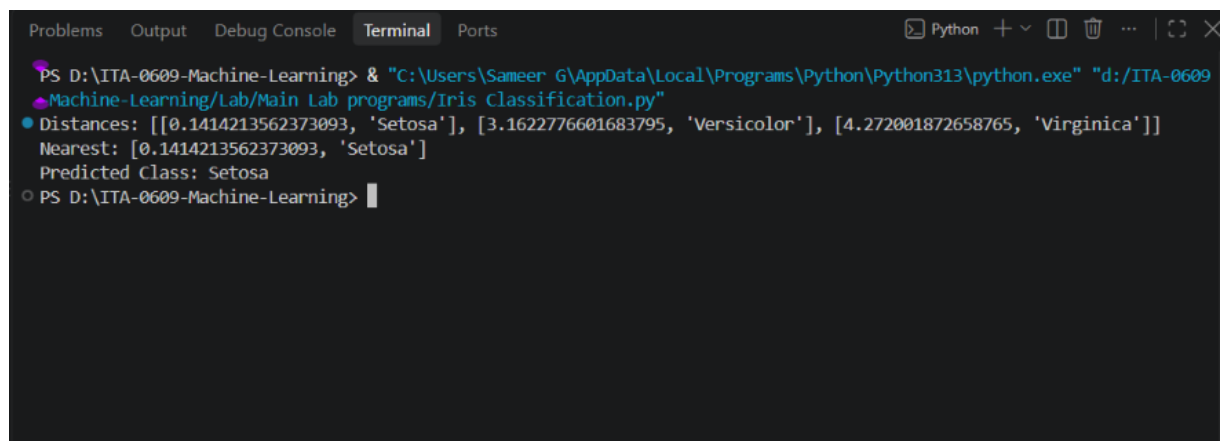
Output

Flower: setosa

Accuracy: approximately 100 %

Result

Thus, the Iris flower was successfully classified using KNN.



```
PS D:\ITA-0609-Machine-Learning> & "C:\Users\Sameer G\AppData\Local\Programs\Python\Python313\python.exe" "d:/ITA-0609-Machine-Learning/Lab/Main Lab programs/Iris Classification.py"  
Distances: [[0.1414213562373093, 'Setosa'], [3.1622776601683795, 'Versicolor'], [4.272001872658765, 'Virginica']]  
Nearest: [0.1414213562373093, 'Setosa']  
Predicted Class: Setosa  
PS D:\ITA-0609-Machine-Learning>
```


13. Car Price Prediction

Aim

To predict the price of a car using Linear Regression.

Algorithm

1. Read car year and price.
2. Train Linear Regression.
3. Provide a new car year.
4. Predict its price.
5. Display the result.

Code

```
from sklearn.linear_model import LinearRegression
```

```
X = [[2018],[2019],[2020],  
     [2021],[2022]]
```

```
y = [4,4.5,5,5.5,6]
```

```
model = LinearRegression()
```

```
model.fit(X,y)
```

```
test = [[2023]]
```

```
price = model.predict(test)
```

```
print("Predicted Price:",  
      price[0],"Lakhs")
```

```
print("R2 Score:",  
      model.score(X,y))
```

Input

Year:

2018 2019 2020 2021 2022

Test Year: 2023

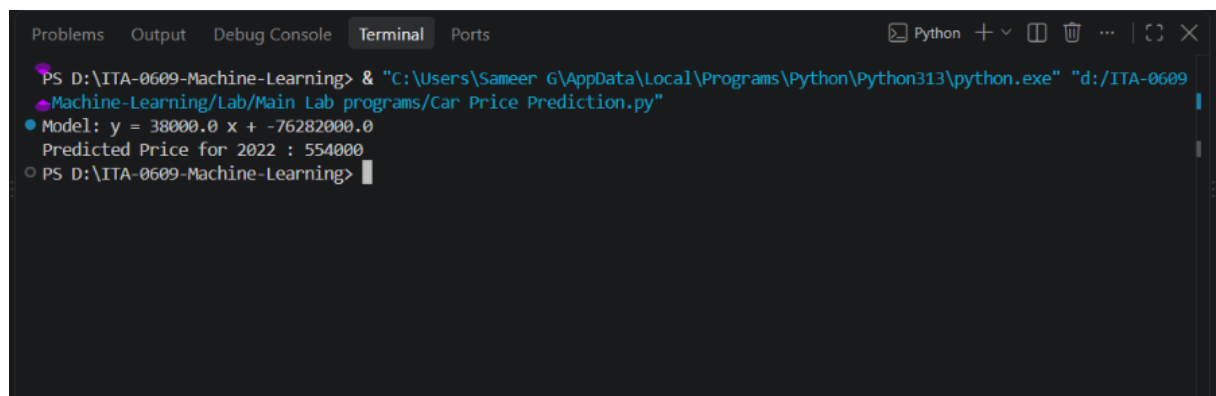
Output

Predicted Price: 6.5 Lakhs

R2 Score: 1.0

Result

Thus, the car price was successfully predicted using Linear Regression.



```
Problems Output Debug Console Terminal Ports
PS D:\ITA-0609-Machine-Learning> & "C:\Users\Sameer G\AppData\Local\Programs\Python\Python313\python.exe" "d:/ITA-0609
Machine-Learning/Lab/Main Lab programs/Car Price Prediction.py"
Model: y = 38000.0 x + -76282000.0
Predicted Price for 2022 : 554000
PS D:\ITA-0609-Machine-Learning>
```

14. House Price Prediction

Aim

To predict house price based on the area using Linear Regression.

Algorithm

1. Read house area and price.
2. Train Linear Regression.
3. Enter the area of a new house.
4. Predict the price.
5. Display R^2 score.

Code

```
from sklearn.linear_model import LinearRegression
```

```
X = [[800],[1000],[1200],  
     [1500],[2000]]
```

```
y = [40,50,60,75,100]
```

```
model = LinearRegression()
```

```
model.fit(X,y)
```

```
test = [[1600]]
```

```
price = model.predict(test)
```

```
print("Predicted Price:",  
      price[0],"Lakhs")
```

```
print("R2 Score:",  
      model.score(X,y))
```

Input

Area = 1600 sq.ft

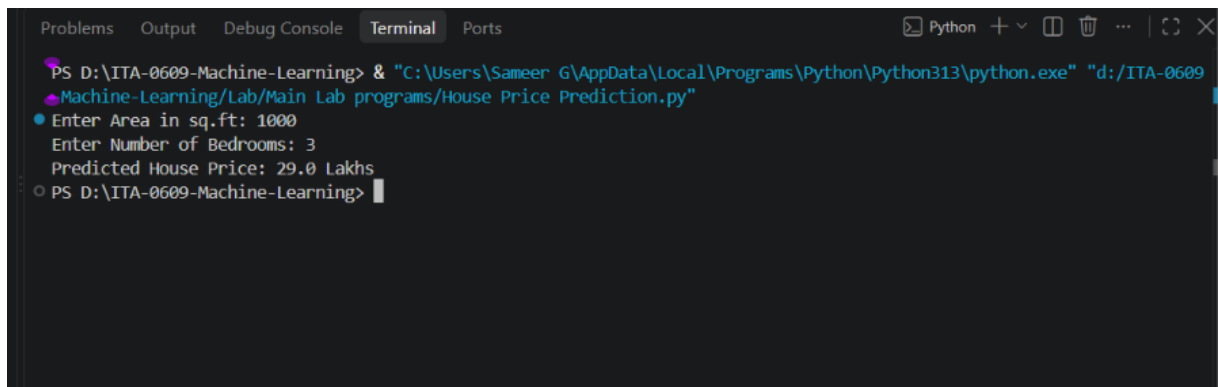
Output

Predicted Price: 80.0 Lakhs

R2 Score: 1.0

Result

Thus, the house price was successfully predicted.



```
Problems Output Debug Console Terminal Ports
PS D:\ITA-0609-Machine-Learning> & "C:\Users\Sameer G\AppData\Local\Programs\Python\Python313\python.exe" "d:/ITA-0609
Machine-Learning/Lab/Main Lab programs/House Price Prediction.py"
Enter Area in sq.ft: 1000
Enter Number of Bedrooms: 3
Predicted House Price: 29.0 Lakhs
PS D:\ITA-0609-Machine-Learning>
```

15. Iris Classification using Naive Bayes

Aim

To classify Iris flowers using the Naive Bayes classifier.

Algorithm

1. Load the Iris dataset.
2. Split it into training and testing sets.
3. Train Gaussian Naive Bayes.
4. Predict the test samples.
5. Calculate accuracy.
6. Display the flower species.

Code

```
from sklearn.datasets import load_iris
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

d = load_iris()

X1,X2,y1,y2 = train_test_split(
    d.data,d.target,
    test_size=.2,
    random_state=42)

model = GaussianNB()
model.fit(X1,y1)

pred = model.predict(X2)

print("Prediction:",pred)
```

```
print("Accuracy:",  
      accuracy_score(y2,pred)*100,"%")
```

```
test = [[5.1,3.5,1.4,0.2]]
```

```
r = model.predict(test)
```

```
print("Flower:",  
      d.target_names[r[0]])
```

Input

Test Flower:

5.1 3.5 1.4 0.2

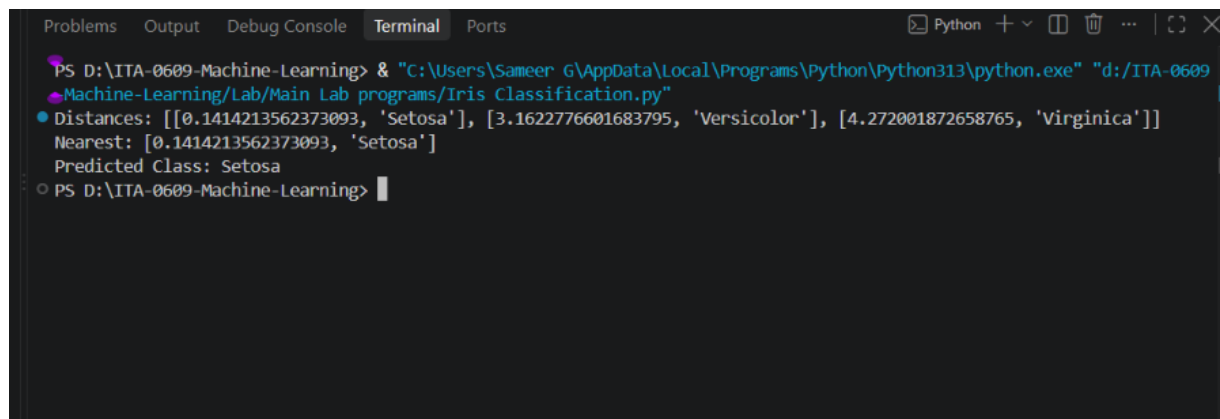
Output

Flower: setosa

Accuracy: approximately 96.67 %

Result

Thus, Iris classification was successfully performed using Naive Bayes.



```
PS D:\ITA-0609-Machine-Learning> & "C:\Users\Sameer G\AppData\Local\Programs\Python\Python313\python.exe" "d:/ITA-0609-Machine-Learning/Lab/Main Lab programs/Iris Classification.py"  
• Distances: [[0.1414213562373093, 'Setosa'], [3.1622776601683795, 'Versicolor'], [4.272001872658765, 'Virginica']]  
  Nearest: [0.1414213562373093, 'Setosa']  
  Predicted Class: Setosa  
○ PS D:\ITA-0609-Machine-Learning>
```

16. Comparison of Classification Algorithms

Aim

To compare different classification algorithms and evaluate their performance.

Algorithm

1. Load the Iris dataset.
2. Split into training and testing data.
3. Train KNN, Naive Bayes and Decision Tree.
4. Predict test data.
5. Calculate accuracy for each.
6. Compare the results.

Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```

```
d = load_iris()
```

```
X1,X2,y1,y2 = train_test_split(
    d.data,d.target,
    test_size=.2,
    random_state=42)
```

```
models = [
    KNeighborsClassifier(3),
    GaussianNB(),
    DecisionTreeClassifier(random_state=42)
```

```
]
```

```
names = ["KNN","Naive Bayes","Decision Tree"]
```

```
for name,model in zip(names,models):
```

```
    model.fit(X1,y1)
```

```
    p = model.predict(X2)
```

```
    print(name,":",
```

```
          accuracy_score(y2,p)*100,"%")
```

Input

Iris dataset

Test size = 20%

Output

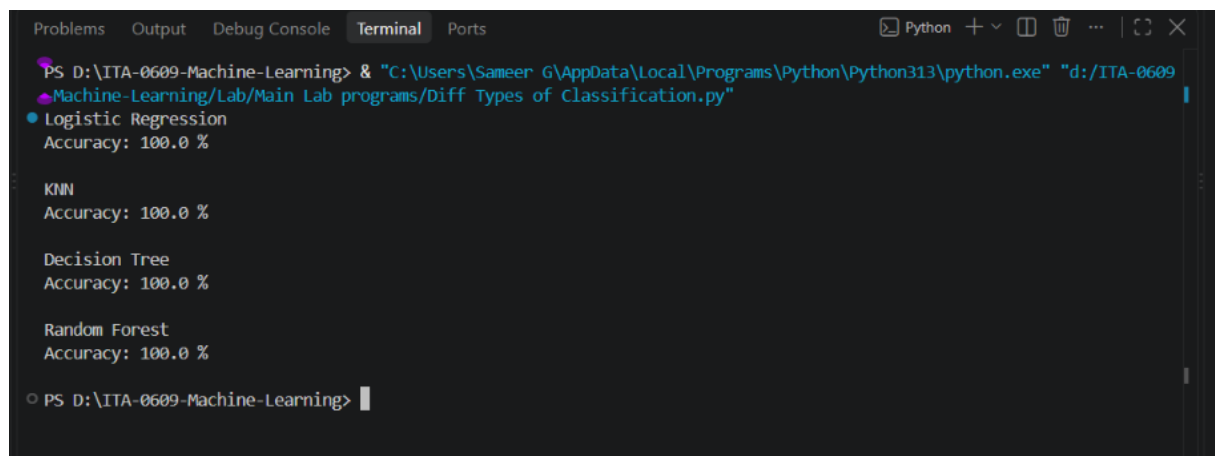
KNN : approximately 100.0 %

Naive Bayes : approximately 96.67 %

Decision Tree : approximately 100.0 %

Result

Thus, different classification algorithms were successfully compared based on accuracy.



```
PS D:\ITA-0609-Machine-Learning> & "C:\Users\Sameer G\AppData\Local\Programs\Python\Python313\python.exe" "d:/ITA-0609-Machine-Learning/Lab/Main Lab programs/Diff Types of Classification.py"
Logistic Regression
Accuracy: 100.0 %

KNN
Accuracy: 100.0 %

Decision Tree
Accuracy: 100.0 %

Random Forest
Accuracy: 100.0 %

PS D:\ITA-0609-Machine-Learning>
```


17. Mobile Price Prediction

Aim

To predict mobile phone price category based on specifications.

Algorithm

1. Read mobile specifications.
2. Assign price categories.
3. Train a Decision Tree.
4. Give new mobile specifications.
5. Predict the price category.
6. Display the result.

Code

```
from sklearn.tree import DecisionTreeClassifier
```

```
X = [  
    [2,3000],  
    [3,3500],  
    [4,4000],  
    [6,4500],  
    [8,5000]  
]
```

```
y = ['Low','Low','Medium',  
     'High','High']
```

```
model = DecisionTreeClassifier(  
    random_state=42)
```

```
model.fit(X,y)
```

```
test = [[6,4500]]
```

```
print("RAM:",test[0][0],"GB")
```

```
print("Battery:",test[0][1],"mAh")
```

```
print("Price Category:",  
      model.predict(test)[0])
```

Input

RAM = 6 GB

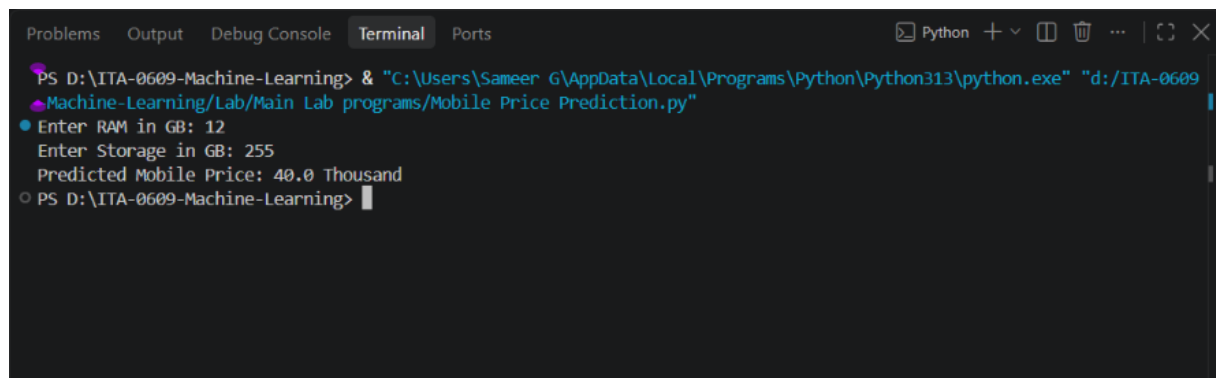
Battery = 4500 mAh

Output

Price Category: High

Result

Thus, the mobile price category was successfully predicted.



```
Problems  Output  Debug Console  Terminal  Ports
PS D:\ITA-0609-Machine-Learning> & "C:\Users\Sameer G\AppData\Local\Programs\Python\Python313\python.exe" "d:/ITA-0609
Machine-Learning/Lab/Main Lab programs/Mobile Price Prediction.py"
Enter RAM in GB: 12
Enter Storage in GB: 255
Predicted Mobile Price: 40.0 Thousand
PS D:\ITA-0609-Machine-Learning>
```

18. Perceptron-based Iris Classification

Aim

To implement a Perceptron classifier for Iris classification.

Algorithm

1. Load Iris data.
2. Select two classes.
3. Split the dataset.
4. Train the Perceptron.
5. Predict test samples.
6. Calculate accuracy.

Code

```
from sklearn.datasets import load_iris
from sklearn.linear_model import Perceptron
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

d = load_iris()

X = d.data[:100]
y = d.target[:100]

X1,X2,y1,y2 = train_test_split(
    X,y,test_size=.2,
    random_state=42)

model = Perceptron(
    random_state=42)

model.fit(X1,y1)
```

```
pred = model.predict(X2)
```

```
print("Prediction:",pred)
```

```
print("Accuracy:",  
      accuracy_score(y2,pred)*100,"%")
```

Input

Iris Setosa and Versicolor data.

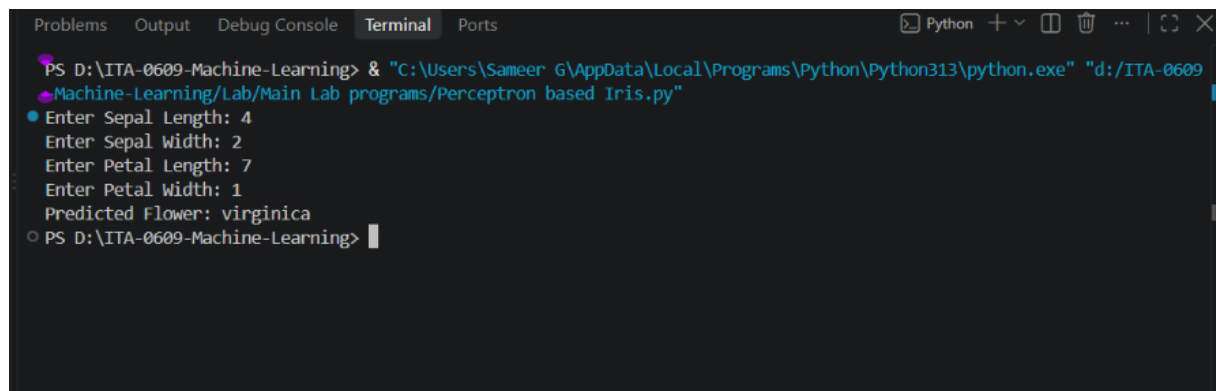
Output

Prediction: [0 1 0 1 ...]

Accuracy: approximately 95–100 %

Result

Thus, Iris classification was successfully performed using a Perceptron.



```
PS D:\ITA-0609-Machine-Learning> & "C:\Users\Sameer G\AppData\Local\Programs\Python\Python313\python.exe" "d:/ITA-0609-Machine-Learning/Lab/Main Lab programs/Perceptron based Iris.py"  
● Enter Sepal Length: 4  
  Enter Sepal Width: 2  
  Enter Petal Length: 7  
  Enter Petal Width: 1  
  Predicted Flower: virginica  
○ PS D:\ITA-0609-Machine-Learning>
```

19. Naive Bayes Bank Loan Prediction

Aim

To predict whether a bank loan should be approved using Naive Bayes.

Algorithm

1. Read income and debt data.
2. Assign loan approval labels.
3. Train Gaussian Naive Bayes.
4. Provide new customer data.
5. Predict loan approval.
6. Display accuracy.

Code

```
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
```

```
X = [
    [50,10],
    [20,15],
    [60,5],
    [25,18],
    [70,4]
]
```

```
y = ['Yes','No','Yes','No','Yes']
```

```
model = GaussianNB()
model.fit(X,y)
```

```
test = [
    [55,7],
```

```
[22,16]
]

pred = model.predict(test)

print("Loan Prediction:",pred)

print("Accuracy:",
      accuracy_score(
          y,model.predict(X))*100,"%")
```

Input

Customer 1:

Income = 55

Debt = 7

Customer 2:

Income = 22

Debt = 16

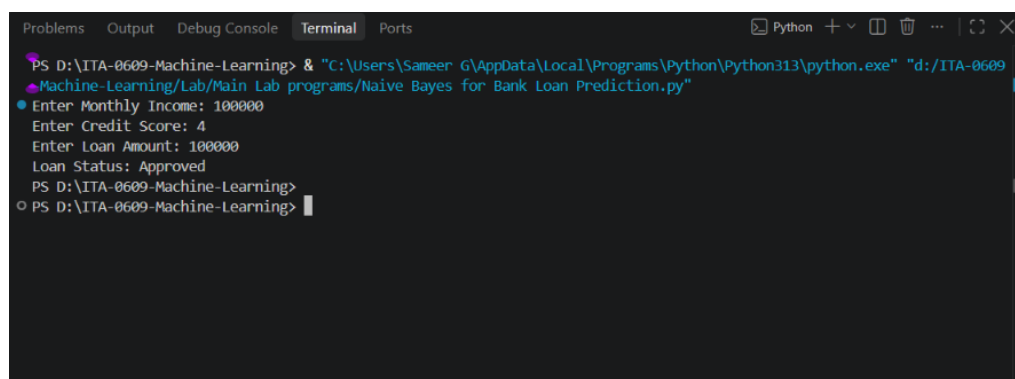
Output

Loan Prediction: ['Yes' 'No']

Accuracy: 100.0 %

Result

Thus, bank loan approval was successfully predicted using Naive Bayes.



```
Problems Output Debug Console Terminal Ports
PS D:\ITA-0609-Machine-Learning> & "C:\Users\Sameer G\AppData\Local\Programs\Python\Python313\python.exe" "d:/ITA-0609
Machine-Learning/Lab/Main Lab programs/Naive Bayes for Bank Loan Prediction.py"
Enter Monthly Income: 100000
Enter Credit Score: 4
Enter Loan Amount: 100000
Loan Status: Approved
PS D:\ITA-0609-Machine-Learning>
PS D:\ITA-0609-Machine-Learning>
```

20. Future Sales Prediction

Aim

To predict future sales using a suitable machine learning algorithm.

Algorithm

1. Read previous sales data.
2. Use time/month as input.
3. Train Linear Regression.
4. Predict sales for the next month.
5. Display predicted sales.

Code

```
from sklearn.linear_model import LinearRegression
```

```
X = [[1],[2],[3],[4],[5],[6]]
```

```
y = [10,20,30,40,50,60]
```

```
model = LinearRegression()
```

```
model.fit(X,y)
```

```
test = [[7]]
```

```
prediction = model.predict(test)
```

```
print("Previous Sales:",y)
```

```
print("Next Month:",test[0][0])
```

```
print("Predicted Sales:",
```

```
    prediction[0])
```

```
print("R2 Score:",
```

```
    model.score(X,y))
```

Input

Previous Sales:

10, 20, 30, 40, 50, 60

Next Month = 7

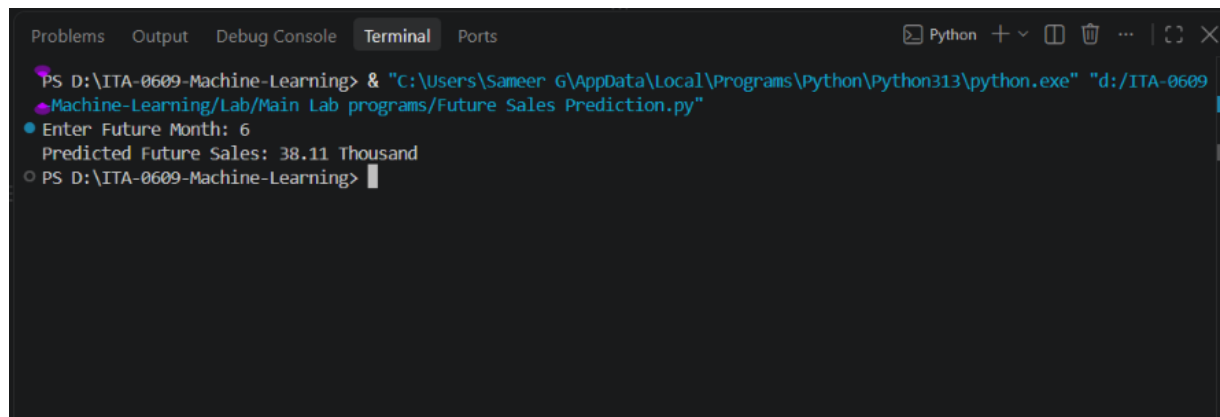
Output

Predicted Sales: 70.0

R2 Score: 1.0

Result

Thus, future sales were successfully predicted using Linear Regression.



```
PS D:\ITA-0609-Machine-Learning> & "C:\Users\Sameer G\AppData\Local\Programs\Python\Python313\python.exe" "d:/ITA-0609-Machine-Learning/Lab/Main Lab programs/Future Sales Prediction.py"
Enter Future Month: 6
Predicted Future Sales: 38.11 Thousand
PS D:\ITA-0609-Machine-Learning>
```